

TimeTracker

AI-Driven Software Development — Project Report

Ali Akbar Baloch

Summer Semester 2026

Contents

1	Implemented Features	3
1.1	Mandatory functional requirements	3
1.2	Additional customer requirements	3
1.3	Mandatory non-functional requirements	4
1.4	Security and reliability hardening	5
2	System Architecture and Design Decisions	5
2.1	High-level architecture	5
2.2	Authentication design	5
2.3	Login rate limiting	6
2.4	Database and persistence	6
2.5	Task soft delete and data integrity	6
2.6	Backend layer structure	7
2.7	Project sharing data model	7
2.8	Task-project many-to-many relationship	7
2.9	Paginated task list	7
2.10	Real-time timer synchronisation via SSE	8
2.11	React Error Boundary	8
2.12	Accumulated timer design	8
2.13	Frontend state management	8
2.14	Performance design	9
2.15	Test strategy	9
3	Used LLMs and AI-based Tools	10
3.1	How Claude Code was used	10
4	Where Did the AI Tools Work Well?	10
4.1	Consistent boilerplate generation	10
4.2	Comprehensive test authoring	11
4.3	Complex cross-cutting features	11
4.4	CI failure diagnosis	11

4.5	UI/UX overhaul	12
5	Where Did the AI Tools Not Work as Intended?	12
5.1	Playwright E2E test flakiness	12
5.2	Incremental bugs introduced after merge	13
5.3	Overly broad initial implementations	13
6	Custom Features	13
6.1	US-026 — Project Time Budgets	14
6.2	US-027 — Task Templates	14
6.3	US-028 — Productivity Analytics	14

1 Implemented Features

TimeTracker is a complete time-tracking application implementing all mandatory requirements, all four additional customer requirements, three custom features, and a set of security and reliability hardening improvements. The following subsections summarise what was built; Sections 2–6 explain the design rationale in depth.

1.1 Mandatory functional requirements

User Management. Users register with an e-mail address, display name, and password. On login the server returns a signed JWT that the browser stores in `localStorage`. A dedicated *Change Password* page lets authenticated users update their credentials without re-registering. Logout revokes the server-side refresh token and clears all local storage.

Time Tracking. A single **Start / Stop** button in the top bar starts and stops a live timer. The elapsed time (HH:MM:SS) is updated every second and persisted in the database (`tasks.start_time`, `tasks.end_time`), so the timer survives a browser reload or a cross-device switch. Completed tasks can be added manually with a custom start and end time, edited (including moving the start time), or soft-deleted (the record is preserved in the database with a `deleted_at` timestamp rather than being removed, protecting the audit trail). An active timer is rehydrated from `GET /api/tasks/active` on every page load.

Projects. Users create projects and optionally nest them as subprojects to arbitrary depth. A task can be associated with one or more projects simultaneously via a `task_projects` join table. The project summary aggregates time recursively: time logged against a subproject is counted towards all ancestor projects. If the same task is linked to two sibling subprojects, it is counted only once at the parent level (set-based deduplication in the service layer). Projects can be renamed, given a budget, or deleted (force-delete cascades to subprojects and disassociates tasks).

Task Overview. The *Tasks* page shows a paginated, filterable list of tasks (20 per page). Previous / Next controls and a “Showing X–Y of Z tasks” indicator appear automatically once the result set exceeds one page. Users can filter by project, date range, description keyword, and (for shared projects) by contributing user. All filters are applied first in the service layer; pagination is then applied to the filtered result so the page count always reflects the active filter. A *Dashboard* page presents today’s and this week’s totals, the running task, and the five most-active projects at a glance in a single API call.

Data Persistence. All data is stored in an H2 file-based database (`timetracker.mv.db`) that survives container restarts. The DDL strategy is `update`, so schema changes are applied incrementally without data loss. The timer state (running task, start time) is stored in the database; there is no reliance on browser storage for timing, so switching browsers or devices shows the correct running time.

1.2 Additional customer requirements

Project Sharing. A project owner can invite other registered users by e-mail. Accepted invitations create a row in `project_members` with role `MEMBER`. Members can view the project, log tasks against it, and see the aggregated totals; only the owner can rename,

delete, or modify budget settings. All write endpoints use `findByIdAndUser` (owner check); read endpoints use `findByIdAndMember` (any member).

Shared Task Overview. On the Tasks page, a *user filter* drop-down lists all members of the current project. Selecting a member shows only their tasks. The backend enforces that both the requesting user and the target user must be members of the project; non-member target requests return 403.

Export. The export button on a project page downloads a CSV file containing every task in the project and its subprojects. Each row includes task description, start time, end time, duration, associated projects, and the contributing user. An optional month filter restricts the export to a specific calendar month.

Time Zones. Each user can set a preferred IANA time zone (e.g. `Europe/Berlin`) on their profile page. All timestamps are stored as UTC in the database; the display layer converts them using `Intl.DateTimeFormat` with the user's stored timezone. A custom `localDateToUtcIso` utility corrects for DST by measuring the actual UTC offset at the target moment rather than the nominal one, preventing a one-hour shift at DST boundaries.

1.3 Mandatory non-functional requirements

Usability. The frontend is a React 19 single-page application with a persistent sidebar, a live timer in the top bar, and a dedicated page for each domain (Dashboard, Tasks, Projects, Analytics, Profile). Frequently performed actions — starting or stopping the timer, creating a task, switching projects — each require one or two clicks. The Dashboard page consolidates today's and this week's totals, the currently running task, and the five most-active projects into a single view served by one API call, so users get an overview without navigating away.

Security. Passwords are hashed with BCrypt. Authentication uses stateless JWTs (1-hour access tokens) paired with server-side refresh tokens stored in a dedicated `refresh_tokens` table. Every protected endpoint requires a valid Bearer token; the Spring Security filter chain rejects unauthenticated requests with 401 before they reach the controller layer. All request DTOs are validated with Jakarta Bean Validation (`@NotBlank`, `@Email`, `@Size`) so malformed input is rejected at the boundary. Spring Data JPA parameterised queries prevent SQL injection. A login rate limiter (10 requests/min per IP by default) protects against brute-force credential attacks.

Performance. The frontend is served as a production-optimised Vite bundle via Nginx (gzip-compressed static assets), so the initial page load is a single small HTML file with hashed JS/CSS chunks that the browser caches across navigations. API responses are served from the local H2 database with no external service calls on the critical path; typical response times on localhost are under 50 ms. The backend Docker image starts and is ready to serve health-check requests in under 10 seconds.

Independence. The entire application — backend, frontend, and database — runs locally with a single command: `docker compose up --build`. No external cloud service or third-party API is required at runtime. The H2 database is embedded in the backend and persisted to a Docker named volume (`timetracker-data`) that survives container restarts. For production use, a `docker-compose.prod.yml` overlay switches the database to a locally-run PostgreSQL container while keeping the same single-command workflow.

1.4 Security and reliability hardening

Beyond the stated requirements, six security and reliability improvements were implemented ([Issue #141](#), [PR #142](#)):

1. **React Error Boundary** — catches unexpected render errors and shows a recovery UI instead of a blank page.
2. **Task soft delete** — preserves the audit trail; deleted tasks remain in the database with a `deleted_at` timestamp.
3. **Login rate limiting** — blocks brute-force credential stuffing by limiting login attempts per IP to 10 per minute (configurable).
4. **JWT refresh tokens** — short-lived access tokens (1 hour) paired with long-lived refresh tokens; the frontend silently renews access tokens on 401 without interrupting the user.
5. **Server-Sent Events timer push** — the backend broadcasts timer start/stop events to all connected browser tabs of the same user, keeping the timer in sync across multiple open windows.
6. **PostgreSQL production profile** — a prod Spring profile and a Docker Compose overlay file enable a one-command switch from H2 to PostgreSQL for production deployments.

2 System Architecture and Design Decisions

2.1 High-level architecture

TimeTracker is a full-stack single-page application structured around a clear separation of concerns: a **Spring Boot 3 / Java 21 backend** that exposes a RESTful JSON API, a **React 19 + Vite frontend** that communicates with it exclusively over HTTP and SSE, and a **file-based H2 database** for development (switchable to PostgreSQL for production). The backend uses Spring Data JPA for the persistence layer, Spring Security 6 for authentication and authorisation, and jjwt 0.12.6 for token generation and validation.

2.2 Authentication design

Authentication is **stateless JWT-based**: on login the backend signs a short-lived access token (1 hour, HMAC-SHA256) and a long-lived refresh token (30 days, stored hashed in the `refresh_tokens` table). The client stores the access token in `localStorage` and attaches it as a `Bearer` header on every request. When a request returns 401, the Axios response interceptor silently calls `POST /api/auth/refresh` with the stored refresh token; if successful, it retries the original request with the new access token — the user never sees the interruption. If the refresh token is expired or revoked, the interceptor clears storage and redirects to `/login`.

Refresh tokens **rotate on every use**: each call to `/api/auth/refresh` invalidates the submitted token and issues a new one. This limits the damage from a stolen token to at most one more use.

`POST /api/auth/logout` accepts the refresh token in the request body and immediately revokes it in the database, so the user cannot silently regain access with a captured token after logging out.

BCrypt at cost factor 10 hashes all stored passwords; the random salt ensures that two identical passwords produce different hashes, neutralising pre-computation attacks.

The JWT signing secret is supplied via the `JWT_SECRET` environment variable and falls back to a built-in development default when not set. The built-in default is acceptable for local development and CI but must be replaced with a secret value in production, because the default is publicly visible in the repository.

2.3 Login rate limiting

A `RateLimitService` maintains a `ConcurrentHashMap` of per-IP sliding windows. Each entry records the request count and window expiry (`nowMs + 60,000 ms`). The default limit is **10 login attempts per IP per minute**, controlled by the `app.rate-limit.login-per-minute` property (overridden to 1000 in Docker Compose and to 1000 in the test profile, so CI E2E tests are not blocked).

The filter uses `request.getRequestURI()` rather than `getServletPath()`, because `getServletPath()` returns an empty string in Spring MockMvc, causing the filter to be skipped in every integration test. This distinction is non-obvious and is documented in `CLAUDE.md` to prevent regression.

2.4 Database and persistence

H2 is used in two modes: **file-based** (`jdbc:h2:file:./data/timetracker`) in development and Docker Compose, and **in-memory** (`jdbc:h2:mem:testdb`) for the test profile. This eliminated every external dependency: a developer can clone the repository, run `docker compose up --build`, and have a fully functional stack within minutes.

A **PostgreSQL production profile** (`application-prod.properties`) is provided for deployments that require a production-grade database. The profile reads connection details from `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, and `DB_PASSWORD` environment variables and sets the Hibernate dialect to `PostgreSQLDialect`. A `docker-compose.prod.yml` overlay adds a PostgreSQL 17 service and wires the variables automatically. Switching from H2 to PostgreSQL requires only setting `APP_PROFILE=prod` in the environment; no application code changes.

The DDL strategy is **create-drop** in tests and **update** in development, so the schema is auto-created on first boot and survives restarts without manual SQL steps.

2.5 Task soft delete and data integrity

Instead of removing rows from the `tasks` table, a `DELETE /api/tasks/{id}` request sets the `deleted_at` column to the current UTC timestamp. The `@SQLRestriction("deleted_at IS NULL")` annotation on the `Task` entity transparently filters deleted rows from every JPA query, including collection fetches, pagination, and `deleteAll()` in tests — without changing any repository or service code.

A subtle consequence: `taskRepository.deleteAll()` in `@BeforeEach` only deletes non-deleted tasks. Soft-deleted tasks remain and their foreign keys point at the user being deleted, causing FK constraint violations during test teardown. The fix adds `@OnDelete(action = OnDeleteAction.CASCADE)` to the `Task.user` mapping and an explicit `ON DELETE CASCADE` foreign-key definition on the `task_id` column of the `task_projects` join table. The same `@OnDelete(CASCADE)` annotation is applied to `RefreshToken.user` for the same reason. These are database-level cascades (DDL), not JPA-level cascades, so they execute regardless of what JPA can see through its filter.

2.6 Backend layer structure

The backend follows a strict four-layer convention:

- **entity/** — JPA entities: `User`, `Task`, `Project`, `ProjectMember`, `TaskTemplate`, `RefreshToken`.
- **repository/** — Spring Data JPA interfaces; method names derive queries; `@Query` used only when derivation is insufficient.
- **service/** — all business logic, the exclusive mutation testing (PITest) target.
- **controller/** — thin REST layer; delegates immediately to the service, enforces HTTP-level concerns only.

2.7 Project sharing data model

Sharing is implemented through a dedicated `project_members` join table with columns (`project_id`, `user_id`, `role`, `joined_at`) and a unique constraint on (`project_id`, `user_id`). The `project.user_id` column was retained as the original owner foreign key for backward compatibility. All read operations use a `findByIdAndMember` query (any member may read); write operations use `findByIdAndUser` (owner only). A `MembershipSeeder` component back-fills `OWNER` rows on application startup for all projects created before the sharing feature was added.

2.8 Task-project many-to-many relationship

Tasks and projects are connected through a `task_projects` join table managed with `@ManyToMany`. This allows a single task to belong to multiple projects simultaneously. Cascading is managed at the database level: an `ON DELETE CASCADE` foreign key on `task_projects.task_id` ensures that soft-deleted task rows (invisible to JPA) are cleaned up when their owner user is deleted.

2.9 Paginated task list

`GET /api/tasks` returns a `PageResponse<TaskResponse>` wrapper instead of a bare array:

```
{ "content": [...], "totalElements": 42,
  "totalPages": 3, "currentPage": 0, "pageSize": 20 }
```

All active filters (project, date range, keyword, user) are applied first in the service layer, and pagination is performed on the already-filtered list. This means the page count always

reflects the current filter rather than the total number of tasks. Page index and size are clamped server-side (minimum 1, maximum 100) so invalid query parameters cannot crash or over-fetch.

On the frontend the *Tasks* page reads `.content` from the response and renders Previous / Next navigation buttons together with a “Showing X–Y of Z tasks” indicator. The controls are hidden entirely when `totalPages` is 1, keeping the UI clean for users with fewer than 20 tasks. Changing any filter resets the page index to 0 so the user always sees the first page of the new result set.

2.10 Real-time timer synchronisation via SSE

A `TimerEventService` holds a `ConcurrentHashMap<String, List<SseEmitter>>` mapping user e-mail to open browser connections. When `TaskService.startTask` or `stopTask` completes, it publishes a Spring application event (`TimerStartedEvent` / `TimerStoppedEvent`). The service listens to these events and sends the appropriate SSE frame to every emitter registered for that user’s e-mail.

On the frontend, `TimerContext.jsx` opens an `EventSource` to `/api/timer/events?token=...` only when a valid JWT is present in `localStorage`. Receiving a `timer-started` event immediately updates the active task state; a `timer-stopped` event clears it. This means that starting a timer in one browser tab is instantly reflected in any other open tab without polling.

2.11 React Error Boundary

An `ErrorBoundary` class component wraps all protected routes in `App.jsx`. When a child throws during render, the boundary’s error lifecycle sets an internal error flag and logs the error to the console via `componentDidCatch`. The fallback UI shows a user-friendly message and a *Try Again* button.

The “Try Again” handler resets `hasError` to `false`. However, the children must be swapped to a non-throwing component *before* the state reset is triggered; otherwise React immediately re-renders with the original throwing children, catches the error again, and sets `hasError` back to `true` before the UI can update. This ordering constraint is tested explicitly in `ErrorBoundary.test.jsx` using `rerender` followed by `fireEvent.click`.

2.12 Accumulated timer design

When a user starts a task template for the second or third time, a new `Task` row is created (preserving a complete audit trail) but the timer is initialised to the sum of all previous completed sessions. This is modelled via a `totalPreviousSeconds` column (bigint, default 0) on the `Task` entity. The frontend timer displays `totalPreviousSeconds + (now - startTime)`, giving the user a continuously growing counter across template restarts without changing the underlying entity semantics.

2.13 Frontend state management

The frontend intentionally avoids Redux or any global state library. State is managed through two React Contexts:

- **AuthContext** — JWT token, user e-mail, display name, and timezone; persisted in `localStorage`.
- **TimerContext** — the active running task and a live elapsed string, rehydrated from `GET /api/tasks/active` on every mount (no browser storage). SSE events from the backend keep the state in sync across multiple tabs.

A **ThemeContext** supports light, dark, and system-preference modes via CSS custom properties on the `<html>` element.

2.14 Performance design

Database indexes on `tasks(user_id, start_time)`, `tasks(user_id, end_time)`, and `projects(user_id, parent_project_id)` cover the hot query paths. All task-fetching queries use `LEFT JOIN FETCH t.projects` to eliminate the N+1 query problem. The Dashboard summary is computed in a single `GET /api/dashboard/summary` call that returns today/week totals, the running task, and the top-5 projects in one response, eliminating any waterfall. The entire stack (cold Docker build included) starts and responds to `/api/health` in under 60 seconds; after that, all page loads and API calls respond in under one second on a local machine.

2.15 Test strategy

The project ships **535 backend tests** and **339 frontend unit tests**, plus 31 Playwright end-to-end spec files (one per user story / NFR). All three suites run in a GitHub Actions CI pipeline.

- **Backend unit tests** (`service/` package): use Mockito with no Spring context; these are the PITest mutation-testing target and are written to kill setter-mutation mutants via `ArgumentCaptor` assertions.
- **Backend integration tests** (`controller/` package): use `@SpringBootTest + MockMvc + @ActiveProfiles("test")` against an in-memory H2 database. Each test class calls `repository.deleteAll()` in `@BeforeEach`; the `@OnDelete(CASCADE)` database constraints ensure this succeeds even when soft-deleted tasks or refresh tokens reference the users being deleted.
- **Frontend unit tests**: Vitest + React Testing Library; all `src/api/*` modules are mocked with `vi.mock`; coverage thresholds are $\geq 90\%$ lines, $\geq 85\%$ functions, $\geq 80\%$ branches.
- **E2E tests**: Playwright in headless Chromium; the shared `auth.ts` helper registers users via API and injects JWTs directly into `localStorage` to bypass the login form. The Docker Compose environment overrides the login rate limit to 1000 per minute (`APP_RATE_LIMIT_LOGIN_PER_MINUTE=1000`) so that 30+ specs running concurrently from the same IP are not blocked.

Coverage gates enforced by CI: JaCoCo $\geq 90\%$ line coverage, PITest $\geq 80\%$ mutation strength, Vitest $\geq 90\%$ line/statement coverage.

3 Used LLMs and AI-based Tools

The entire project was developed using a single AI tool: **Claude Code** (Anthropic’s CLI agent) running **Claude Sonnet 4.6**, accessed through the VS Code extension. No other LLM or AI-based tool was used.

3.1 How Claude Code was used

Claude Code was used as an **interactive pair-programming agent** for every phase of development:

- **Architecture and design** — discussing and deciding on the layered backend structure, JWT statelessness, H2 file-persistence, the many-to-many task-project model, refresh-token rotation strategy, and soft-delete cascade design before writing a single line of code.
- **Feature implementation** — writing JPA entities, Spring service and controller classes, React page components, CSS, and Axios API wrappers for all 28 user stories, 4 non-functional requirements, and 6 security hardening improvements.
- **Test authoring** — generating the full suite of 535 JUnit/Mockito backend tests and 339 Vitest/React Testing Library frontend tests, covering unit, integration, and component levels, including reflection-based tests for private SSE callback wiring.
- **E2E test authoring** — writing all 31 Playwright spec files (one per user story/NFR), including the shared `auth.ts` helper that registers users via API and injects JWT directly into `localStorage` to bypass the login form.
- **CI pipeline design and debugging** — setting up the GitHub Actions pipeline (backend, frontend, and Playwright system-test jobs in Docker), diagnosing CI failures, and iterating on fixes.
- **Bug investigation and fixing** — all fix PRs (analytics year sync, accumulated timer, NFR-003 layout overflow, Playwright strict-mode violations, rate-limiter path matching, SSE callback reflection, Error Boundary try-again ordering) were diagnosed and resolved through conversations with the agent.
- **Documentation** — writing issue descriptions, PR summaries, the README (test table, architecture notes, JWT secret explanation), and this report.

The `CLAUDE.md` file at the repository root served as a standing briefing document for the agent. It encoded process rules (one issue per PR, CI must never fail, commit style), the full architecture description, and a growing list of design decisions so that every new conversation started with full project context without needing re-explanation.

4 Where Did the AI Tools Work Well?

4.1 Consistent boilerplate generation

The most immediate win was the elimination of boilerplate. Each user story followed the same pattern: entity → repository → service → controller → DTO → frontend page. Claude

Code understood this convention from `CLAUDE.md` and generated all layers coherently in a single turn, including field validation, error handling, and the correct Spring Security integration. What would have taken hours of repetitive typing per story was reduced to minutes.

4.2 Comprehensive test authoring

Claude Code produced test suites that went well beyond happy-path coverage. For example, `TaskServiceTest` uses `ArgumentCaptor` to verify the exact fields written to the database on every save call, specifically to kill setter-mutation PITest mutants. The agent understood that PITest kills mutations by detecting when a field is set to the wrong value, and wrote tests that specifically validated each field assignment — a non-trivial testing strategy. The final suite achieves $\geq 90\%$ JaCoCo line coverage, $\geq 80\%$ PITest mutation-test strength, and $\geq 90\%$ Vitest statement coverage.

The security hardening tests were particularly thorough. The `TimerEventServiceTest` uses Java reflection to access the private `completionCallback` field of `ResponseBodyEmitter` and invokes it directly, because `SseEmitter.complete()` does not trigger `onCompletion` callbacks when called outside a real HTTP handler. The agent identified this Spring internals subtlety and produced the reflection-based test rather than accepting an untested code path.

4.3 Complex cross-cutting features

The project-sharing feature required coordinating changes across the entity model, four repository methods, two service classes, access-control guards in every controller, the frontend project list, project detail page, and the invitation form. Claude Code designed the full `project_members` table approach, wrote the `MembershipSeeder` to back-fill ownership rows for pre-existing data, and produced the integration test suite in one coherent turn. The breadth of that change (touching 12+ files) would have been highly error-prone if implemented manually.

Similarly, the JWT refresh-token feature required coordinating a new JPA entity with a database-level cascade, a new service with rotate-on-use semantics, a new controller endpoint, and a frontend Axios interceptor with retry logic — all without breaking any of the 28 existing Playwright specs. The agent kept all these constraints in mind simultaneously.

4.4 CI failure diagnosis

When the CI pipeline failed, Claude Code diagnosed root causes from raw error output with high accuracy. Two examples from the hardening story:

1. **Rate-limiter filter path** — The `LoginRateLimitFilter` called `getServletPath()`, which returns `""` in `MockMvc` rather than the real path, so the filter never intercepted the login endpoint in integration tests. The agent identified the Spring distinction and applied the one-character fix.
2. **E2E rate-limit exhaustion** — 30+ Playwright specs each called `POST /api/auth/login` via `registerUser()` in `beforeAll`. Running concurrently from the same CI IP address exhausted the 10/min limit; subsequent specs received 429, stored `undefined` as the JWT, and every API call returned 401. The agent

traced the cascade from the `auth.ts` helper through to the failing assertions and diagnosed the single-line fix: `add APP_RATE_LIMIT_LOGIN_PER_MINUTE=1000` to `docker-compose.yml`.

4.5 UI/UX overhaul

The UI/UX redesign produced a complete CSS custom-property token system, a light/dark/system theme switcher, skeleton shimmer loaders, a toast notification system, a fixed sidebar with mobile hamburger menu, route transition animations, and an illustrated SVG empty state — all in a single large PR. The visual result was production-quality, not a rough draft, and required no manual CSS iteration.

5 Where Did the AI Tools Not Work as Intended?

5.1 Playwright E2E test flakiness

Writing Playwright end-to-end tests proved to be the most challenging area. The first full E2E run on CI failed with 30 test failures across 14 spec files. The root causes were varied and subtle:

- **Strict-mode violations:** Playwright's strict mode throws when a locator matches more than one element. A success toast (`role="alert"`) was still in the DOM when a subsequent error `<p role="alert">` appeared, causing `getByRole('alert')` to resolve to two elements. The fix required adding `.filter({ hasText: /pattern/i })`.
- **Timing races:** `waitForResponse` fires at the network layer before the browser's JavaScript `fetch().then()` callback runs and React re-renders. Reading `rows.count()` immediately after the event returned the pre-update DOM count. The fix was to use `expect(rows).not.toHaveCount(oldCount, { timeout: 8_000 })` to poll until the DOM reflected the new state.
- **Time-of-day-dependent failures:** Three tests used midnight UTC as a window boundary. Tasks created about an hour before the test ran appeared outside the window during the first hour of a UTC day. The fix replaced the absolute boundary with a relative 2-hour window.
- **DOM detach after React re-render:** Stopping a timer triggered two rapid state updates that briefly unmounted the start button. Playwright's `click()` kept retrying until the 30-second timeout. The fix was to stop the timer via the API in `beforeEach` before navigating to the page.

The first-generation tests were generated without a full mental model of the browser event loop and React's asynchronous reconciliation. Three follow-up PRs were needed before the full suite passed reliably.

Adaptation: providing the agent with the exact CI log output (including which locator failed, what elements were found, and the stack trace) on each iteration was the key to getting accurate fixes. Vague prompts produced generic solutions; specific prompts with the exact Playwright error and component code produced targeted fixes.

5.2 Incremental bugs introduced after merge

Several features were implemented correctly at a high level but contained subtle logic errors that only manifested later:

- **Analytics year boundary bug:** the weekly-pattern service used `year >= currentYear` to decide the upper time bound, causing future years to display current-year data. The correct logic is `to = min(now, yearEnd)`, with `from = yearStart` for future years forcing `from > to` so the query returns zero rows. This required two separate PRs to fully fix.
- **Accumulated timer omission:** the initial Task Templates implementation always initialised the timer to zero, ignoring previous sessions. The `totalPreviousSeconds` column and accumulation logic were added in a subsequent PR after the templates feature was already merged.
- **NFR-003 layout overflow:** the `.page-content` CSS class and the outer wrapper `div` in `Layout.jsx` both applied `margin-left: var(--sidebar-w)`, doubling the offset to 448 px instead of 224 px and pushing the dashboard content well beyond the 1024 px viewport. Two separate commits were needed: removing the duplicate margin, then changing `1fr` to `minmax(0, 1fr)` in the CSS grid to prevent min-content expansion.
- **SseEmitter callback not triggered in tests:** `SseEmitter.complete()` does not invoke `onCompletion` callbacks when called outside a real HTTP handler. The test for emitter cleanup was written using `emitter.complete()`, which silently did nothing, and so the test always passed even when the cleanup was not wired. The correct approach — discovered after the test was seen to be vacuous — is to reflect on `ResponseBodyEmitter.completionCallback` and call it directly.

Adaptation: in each case, the agent fixed the issue correctly once given the CI failure output and the specific file. Extending the CLAUDE.md “Key Design Decisions” section with the root cause and fix pattern for each class of bug prevented the same mistake from recurring in subsequent PRs.

5.3 Overly broad initial implementations

For several features, the agent produced working code that was more complex than needed. The analytics service originally implemented three separate repository query methods and a custom aggregation loop. The simpler final design reuses the existing repository method and performs all grouping in Java streams, which meant zero new repository methods and no new database indexes. Getting to this simpler design required explicitly asking “can this use the existing repository method instead of a new one?” — the first-pass answer was always to introduce new infrastructure.

6 Custom Features

Three custom features were designed and implemented beyond the mandatory project requirements.

6.1 US-026 — Project Time Budgets

[Issue #63](#) · [PR #70](#)

A project can optionally carry a `budgetHours` value (a positive decimal, e.g. `40.0`). When set, the project detail page, project list, and dashboard top-projects list all display a colour-coded progress bar: green below 80% usage (`ON_TRACK`), orange at 80–99% (`WARNING`), and red at or above 100% (`OVER_BUDGET`).

Key design choices:

- The `budgetHours` column is nullable `Double` on the `Project` entity. A null value means no budget is enforced and no progress bar is rendered.
- Budget usage is an **all-time aggregate** across the full project subtree and all members, intentionally independent of the date-range filter on the project summary.
- `computeBudgetStatus` is a `static` method on `ProjectService`, shared with `DashboardService` so both can compute budget status without creating a circular dependency between the two services.

6.2 US-027 — Task Templates

[Issue #64](#) · [PR #71](#) · [PR #128](#)

A `TaskTemplate` entity stores a user-defined name, an optional description, and optional project associations. Clicking **Start** on a template creates a new running task pre-populated with the template’s description and project links. Starting the same template multiple times accumulates time across sessions: the timer begins from the sum of all previous completed sessions for that description rather than from zero.

Key design choices:

- Templates are **strictly private**; ownership is enforced via `findByIdAndUser`, returning 404 (not 403) to avoid leaking template existence to other users.
- The `startFromTemplate` method delegates to the existing `TaskService.startTask` overload rather than duplicating the 409-if-running check.
- Project associations on a template are resolved with `findByIdAndMember`, preventing users from associating arbitrary project IDs with their templates.
- The accumulated-timer design keeps each session as a **separate Task row** (complete audit trail) while showing the user a continuously growing total via `totalPreviousSeconds + (now - startTime)`.

6.3 US-028 — Productivity Analytics

[Issue #65](#) · [PR #72](#) · [PR #132](#)

A dedicated Analytics page offers three visualisations:

1. **Activity Heatmap** — a GitHub-style 52–53-week grid (7 rows × N columns, Monday-first). Each cell represents one calendar day colour-coded in four levels based on its

share of the year’s maximum tracked time. A year selector lets users browse historical data; future years return an empty grid by design.

2. **Average Hours by Day of Week** — a bar chart showing average tracked seconds per weekday (Mon–Sun) over the last 12 weeks of the selected year. The average is `total / weeks` (not `total / days-with-data`), giving a true per-week mean.
3. **Shared Project Breakdown** — for each shared project where at least two members tracked time in the window, a ranked list of contributor bars showing each person’s share as a percentage. Projects with only one contributor or zero total seconds are excluded.

Key design choices:

- All three endpoints reuse the existing repository query method and the `(user_id, start_time)` composite index — no new database indexes were required.
- Future years return zero results by design: `to = min(now, yearEnd)` means `from = yearStart > to`, so the `BETWEEN` clause returns no rows.
- The frontend heatmap cell array is built from a flat list padded to complete weeks using `(Jan1.getDay() + 6) % 7` (converts JavaScript’s Sunday=0 to Monday=0), rendered via `grid-auto-flow: column` so week columns emerge naturally from a row-major array.

References